

# Data Mining



## Softmax, Logistic Regression with Regularization

# Softmax

# Why Do We Need Softmax?

Binary logistic regression handles:

$$y \in \{0, 1\}$$

But what if:

$$y \in \{1, 2, 3, \dots, K\}$$

Examples:

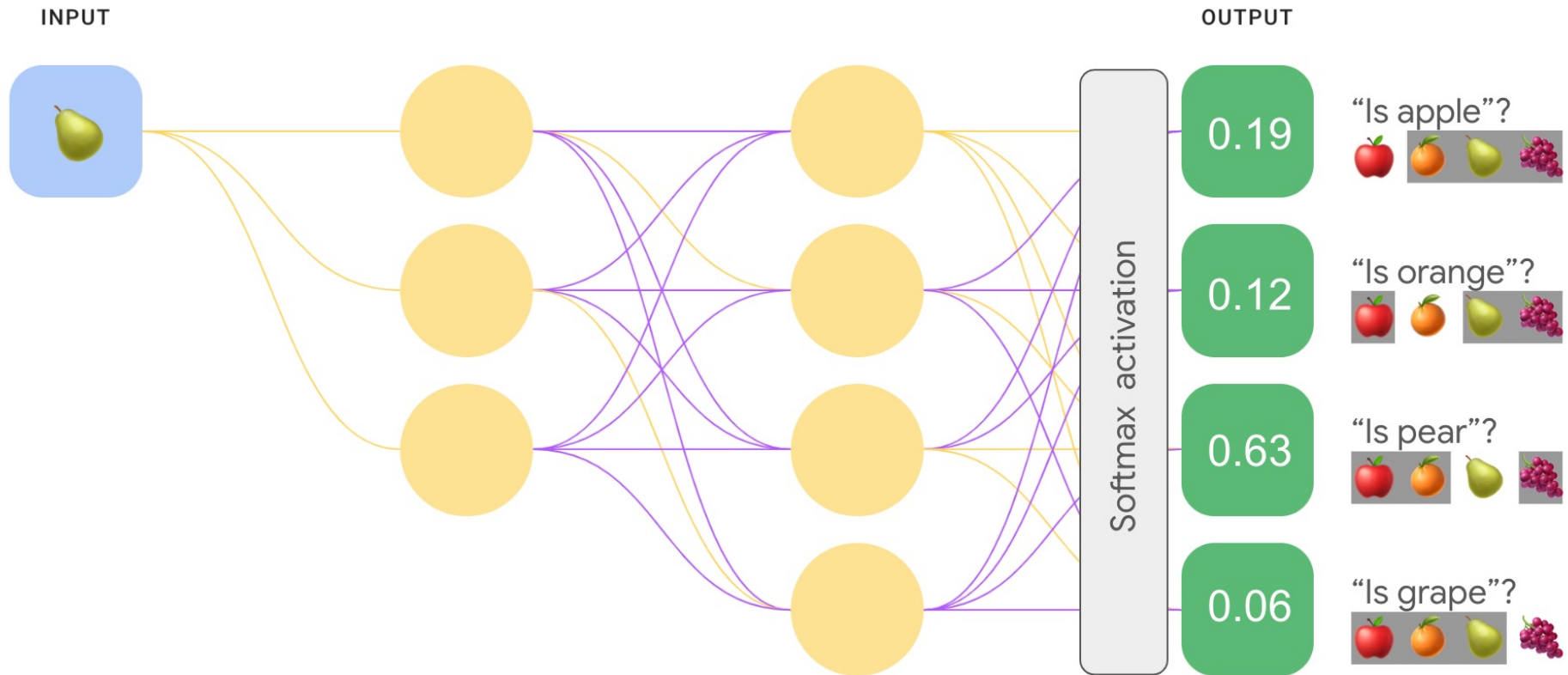
- Topic classification (20 newsgroups)
- Digit recognition (0–9)
- Sentiment: negative / neutral / positive

Binary sigmoid only gives **one probability**.

We now need:

$$P(y = k \mid x), \quad k = 1, \dots, K$$

# Why Do We Need Softmax?



# Model Structure

For each class  $k$ , we learn a separate linear score:

$$z_k = \beta_{k0} + \beta_k^T x$$

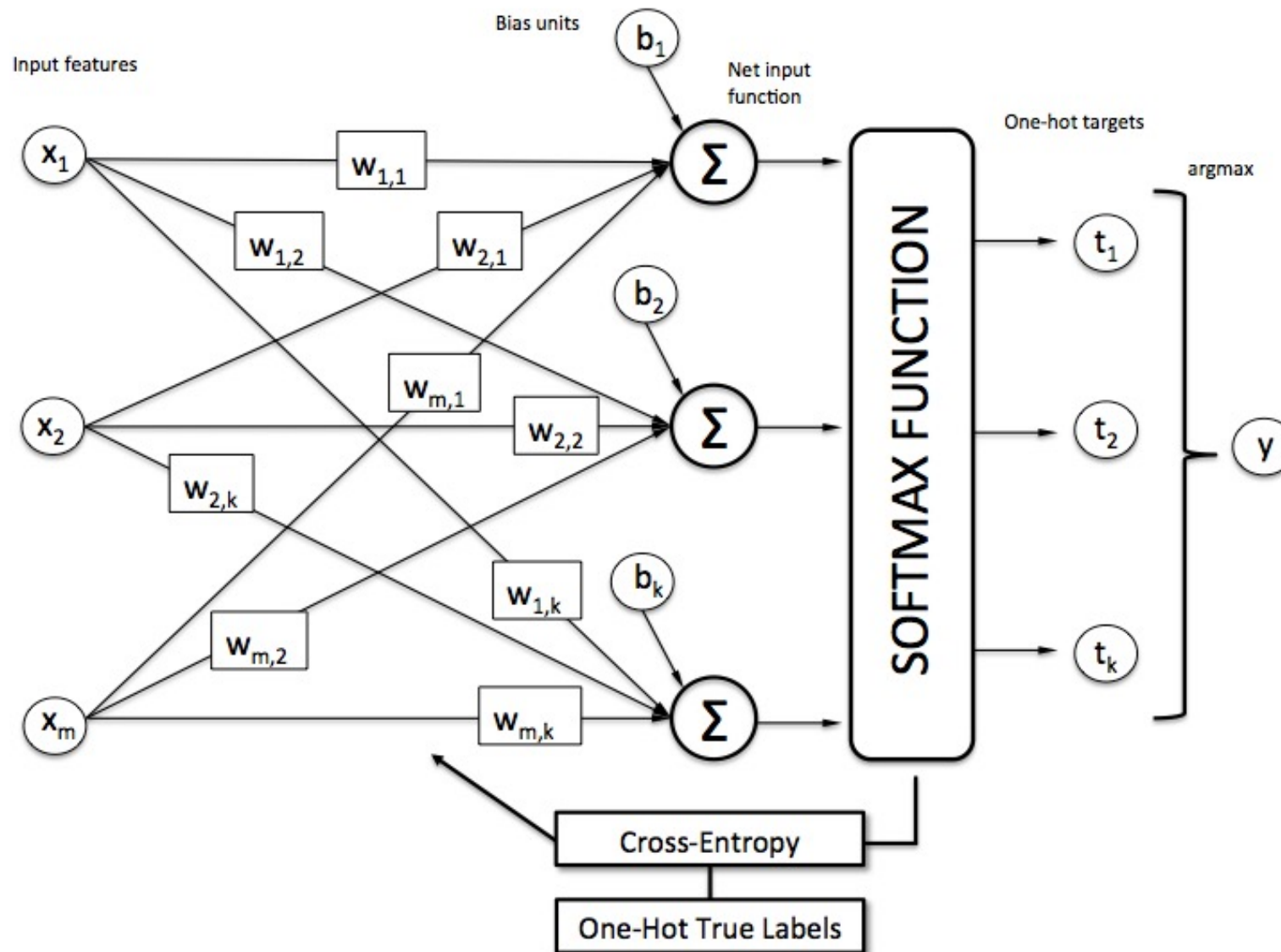
So for a K-class problem, we have:

- K intercepts
- K coefficient vectors

We now have a **vector of logits**:

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix}$$

# Model Structure



# Softmax Function

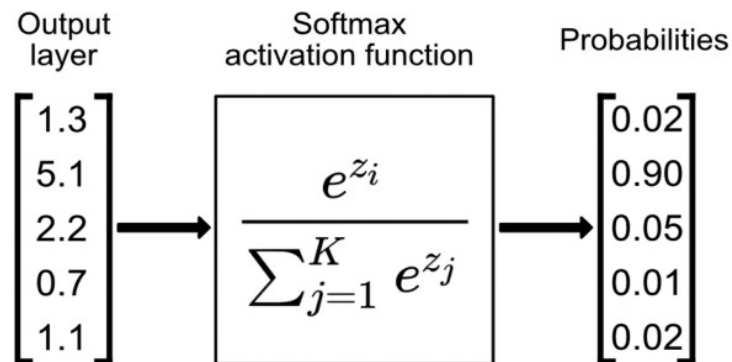
Instead of sigmoid, we use:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

This ensures:

- All probabilities  $\geq 0$
- Sum to 1

$$\sum_{k=1}^K P(y = k \mid x) = 1$$



# Why Exponentials?

Why exponentiate?

1. Keeps probabilities positive
2. Larger  $z_k \rightarrow$  exponentially larger probability
3. Makes optimization smooth & convex

Prediction:

$$\hat{y} = \arg \max_k P(y = k \mid x)$$

Since exponential is monotonic:

$$\arg \max_k P(y = k \mid x) = \arg \max_k z_k$$

Pick class with largest linear score.



# Loss Function (Multiclass Cross-Entropy)

Binary loss:

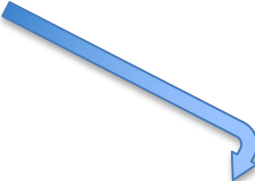
$$-y \log p - (1 - y) \log(1 - p)$$

Multiclass loss:

$$J = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log p_{ik}$$

Where:

- $y_{ik} = 1$  if sample  $i$  belongs to class  $k$
- Otherwise 0


$$\left\{ \begin{array}{l} L^{(i)} = -\sum_{k=1}^K y_{ik} \log p_{ik} \\ J = \frac{1}{N} \sum_{i=1}^N L^{(i)} \end{array} \right.$$

# Softmax Gradient Descent(1)

Softmax regression has this structure:

$$x \rightarrow z = Wx + b \rightarrow p = \text{softmax}(z) \rightarrow L$$

So the loss depends on:

- parameters  $W, b$
- but indirectly through  $z$

Formally:

$$L = L(p(z(W, b)))$$

# Softmax Gradient Descent(2)

We break the derivative into stages:

$$\frac{\partial L}{\partial w_k} = \frac{\partial L}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_k}$$

Instead of doing everything at once.

$$L = - \sum_{j=1}^K y_j \log(p_j)$$

$$p_j = \frac{e^{z_j}}{\sum_{m=1}^K e^{z_m}} \quad \log(p_j) = \log \left( \frac{e^{z_j}}{\sum_m e^{z_m}} \right) = z_j - \log \left( \sum_m e^{z_m} \right)$$

$$L = - \sum_j y_j \left( z_j - \log \sum_m e^{z_m} \right) = - \sum_j y_j z_j + \sum_j y_j \log \sum_m e^{z_m}$$

# Softmax Gradient Descent(3)

$$\begin{aligned} L &= - \sum_j y_j \left( z_j - \log \sum_m e^{z_m} \right) = - \sum_j y_j z_j + \underbrace{\sum_j y_j}_{1} \log \sum_m e^{z_m} \\ &= - \sum_j y_j z_j + \log \sum_m e^{z_m} \end{aligned}$$

Derivative wrt  $z_k$ :

$$\begin{aligned} &\downarrow \qquad \qquad \downarrow \\ &-y_k \qquad \frac{e^{z_k}}{\sum_m e^{z_m}} \\ &\qquad \qquad \downarrow \\ &\qquad \qquad p_k \end{aligned}$$

$$\frac{\partial L}{\partial z_k} = p_k - y_k$$

# Softmax Gradient Descent (4)

Because:

$$z_k = w_k^T x + b_k$$

So:

$$\frac{\partial z_k}{\partial w_k} = x$$

Thus:

$$\frac{\partial L}{\partial w_k} = \frac{\partial L}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_k} = (p_k - y_k)x$$

$$\frac{\partial L}{\partial b_k} = p_k - y_k$$

The update rule becomes:

$$w_k \leftarrow w_k - \alpha(p_k - y_k)x$$

$$b_k \leftarrow b_k - \alpha(p_k - y_k)$$

So:

- If model overpredicts class  $k \rightarrow p_k - y_k > 0 \rightarrow$  subtract  $x$
- If underpredicts  $\rightarrow$  add  $x$

# A 3-class Toy Example

## 1 sample, 3 classes

- Number of classes:  $K = 3$
- Number of features:  $d = 2$
- One sample:

$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad y = \text{class 0} \Rightarrow \mathbf{y} = [1, 0, 0]$$

- Learning rate:  $\alpha = 0.1$

## Initialization (before Iteration 1)

Start with all zeros:

$$W^{(0)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad b^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

# Iteration 1

Compute logits  $z = [0, 0, 0]$

Softmax probabilities  $p = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$

Compute errors  $(p_k - y_k)$  True class is 0, so  $y = [1, 0, 0]$ :

- class 0:  $\frac{1}{3} - 1 = -\frac{2}{3}$
- class 1:  $\frac{1}{3} - 0 = \frac{1}{3}$
- class 2:  $\frac{1}{3} - 0 = \frac{1}{3}$

## Gradients

Multiply each scalar error by  $x = [2, 1]$ :

- $\nabla w_0 = (-\frac{2}{3})[2, 1] = [-\frac{4}{3}, -\frac{2}{3}]$
- $\nabla w_1 = (\frac{1}{3})[2, 1] = [\frac{2}{3}, \frac{1}{3}]$
- $\nabla w_2 = (\frac{1}{3})[2, 1] = [\frac{2}{3}, \frac{1}{3}]$

Bias grads:

$$\nabla b = [-\frac{2}{3}, \frac{1}{3}, \frac{1}{3}]$$

# Iteration 1 (cont.)

Update parameters ( $\alpha = 0.1$ )

$$w_k^{(1)} = w_k^{(0)} - 0.1 \nabla w_k, \quad b_k^{(1)} = b_k^{(0)} - 0.1 \nabla b_k$$

So:

- $w_0^{(1)} = [0.1333, 0.0667], b_0^{(1)} = 0.0667$
- $w_1^{(1)} = [-0.0667, -0.0333], b_1^{(1)} = -0.0333$
- $w_2^{(1)} = [-0.0667, -0.0333], b_2^{(1)} = -0.0333$



# Iteration 2

**Compute logits**  $z_k = w_k^T x + b_k$

Using  $x = [2, 1]$ :

- Class 0:

$$z_0 = [0.1333, 0.0667] \cdot [2, 1] + 0.0667 = (0.2667 + 0.0667) + 0.0667 \approx 0.4000$$

- Class 1:

$$z_1 = [-0.0667, -0.0333] \cdot [2, 1] - 0.0333 = (-0.1333 - 0.0333) - 0.0333 \approx -0.2000$$

- Class 2: same  $\approx -0.2000$

So:

$$z \approx [0.4, -0.2, -0.2]$$

**Softmax probabilities**

$$e^{0.4} \approx 1.4918 \quad e^{-0.2} \approx 0.8187 \quad S = 1.4918 + 0.8187 + 0.8187 \approx 3.1292$$

$$p_0 \approx 1.4918/3.1292 = 0.4767, \quad p_1 \approx 0.2617, \quad p_2 \approx 0.2617$$

# Iteration 2 (Cont.)

## Errors ( $p_k - y_k$ )

- class 0:  $0.4767 - 1 = -0.5233$
- class 1:  $0.2617 - 0 = 0.2617$
- class 2:  $0.2617 - 0 = 0.2617$

## Gradients

Multiply by  $x = [2, 1]$ :

- $\nabla w_0 = (-0.5233)[2, 1] = [-1.0466, -0.5233]$
- $\nabla w_1 = (0.2617)[2, 1] = [0.5233, 0.2617]$
- $\nabla w_2 = (0.2617)[2, 1] = [0.5233, 0.2617]$

Bias grads:

$$\nabla b \approx [-0.5233, 0.2617, 0.2617]$$

# Iteration 2 (Cont.)

## Update again

- $w_0^{(2)} = w_0^{(1)} - 0.1 \nabla w_0 \approx [0.1333, 0.0667] + [0.1047, 0.0523] = [0.2380, 0.1190]$
- $w_1^{(2)} \approx [-0.0667, -0.0333] - [0.0523, 0.0262] = [-0.1190, -0.0595]$
- $w_2^{(2)} \approx [-0.1190, -0.0595]$

## Biases:

- $b_0^{(2)} \approx 0.0667 + 0.0523 = 0.1190$
- $b_1^{(2)} \approx -0.0333 - 0.0262 = -0.0595$
- $b_2^{(2)} \approx -0.0595$

# Comparison to One-vs-Rest (OVR)

- **One-vs-Rest** (`LogisticRegression(multi_class="ovr")`)
  - Train K separate binary classifiers
  - Each predicts “class k vs others”
  - Probabilities not globally normalized
- **Softmax (Multinomial)**  
`LogisticRegression(multi_class="multinomial")`
  - Single model
  - Joint normalization
  - Probabilities consistent

# Connection to Binary Logistic

When  $K = 2$ :

$$P(y = 1) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}}$$

$$e^{z_0} + e^{z_1} = e^{z_1} (e^{z_0 - z_1} + 1)$$

$$\begin{aligned} P(y = 1) &= \frac{e^{z_1}}{e^{z_1} (e^{z_0 - z_1} + 1)} = \frac{1}{e^{z_0 - z_1} + 1} = \frac{1}{1 + e^{z_0 - z_1}} \\ &= \frac{1}{1 + e^{-(z_1 - z_0)}} \end{aligned}$$

That's exactly the sigmoid form:

$$\sigma(t) = \frac{1}{1 + e^{-t}} \quad \text{with} \quad t = z_1 - z_0$$

# Regularization in Softmax

# Why Regularization in Softmax?

- Softmax model has many parameters:
  - Number of parameters =  $K \times d$
  - $K$  = number of classes
  - $d$  = number of features
- High-dimensional data → risk of:
  - overfitting
  - unstable predictions
  - large coefficients
- Main idea:
  - Add a penalty to control model complexity

# Softmax + Cross-Entropy Loss

## Softmax Loss (Recap)

$$J = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log p_{ik}$$

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad z_k = w_k^T x + b_k$$

- Penalizes wrong predictions
- Penalizes overconfident wrong predictions more



# Add Regularization

**L2 (Ridge):**

$$J = \text{loss} + \lambda \sum_k \|w_k\|_2^2$$

**L1 (Lasso):**

$$J = \text{loss} + \lambda \sum_k \|w_k\|_1$$

# How Regularization Changes Learning

Without regularization:

$$\frac{\partial L}{\partial w_k} = (p_k - y_k)x$$

With L2:

L2 → smooth shrinkage

$$\frac{\partial J}{\partial w_k} = (p_k - y_k)x + 2\lambda w_k$$

With L1:

L1 → pushes weights to zero

$$\frac{\partial J}{\partial w_k} = (p_k - y_k)x + \lambda \cdot \text{sign}(w_k)$$

$$\text{sign}(w_{kj}) = \begin{cases} +1 & w_{kj} > 0 \\ 0 & w_{kj} = 0 \\ -1 & w_{kj} < 0 \end{cases}$$

# Geometric Intuition

- **Why L1 Creates Sparsity**
  - L2 constraint → **circle / sphere**
  - L1 constraint → **diamond shape**
- **Result:**
  - L2 → small weights
  - L1 → many weights = 0

# In Softmax (Multiclass)

- Regularization in Multiclass Setting
  - Each class has its own:  $w_k$

Regularization acts **per class**:

– **With L1:**

- Different classes select different features
- Sparse models per class

– **With L2:**

- All features kept
- weights are smaller

# Demo (Digits + Noise Example)

- **Setup:**
  - 64 real pixel features
  - +2000 noise features
- **Observations:**

| Model         | Behavior             |
|---------------|----------------------|
| Plain Softmax | overfits noise       |
| L2            | reduces noise impact |
| L1            | removes noise → best |